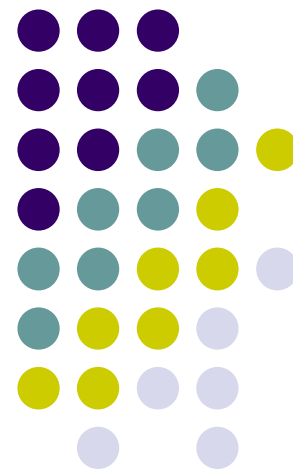
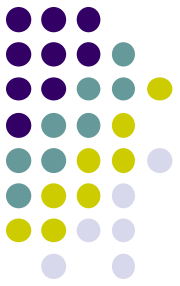


# L16: 最短路 II

吉林大学计算机学院  
谷方明

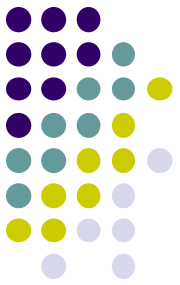
fmgu2002@sina.com





# 学习目标

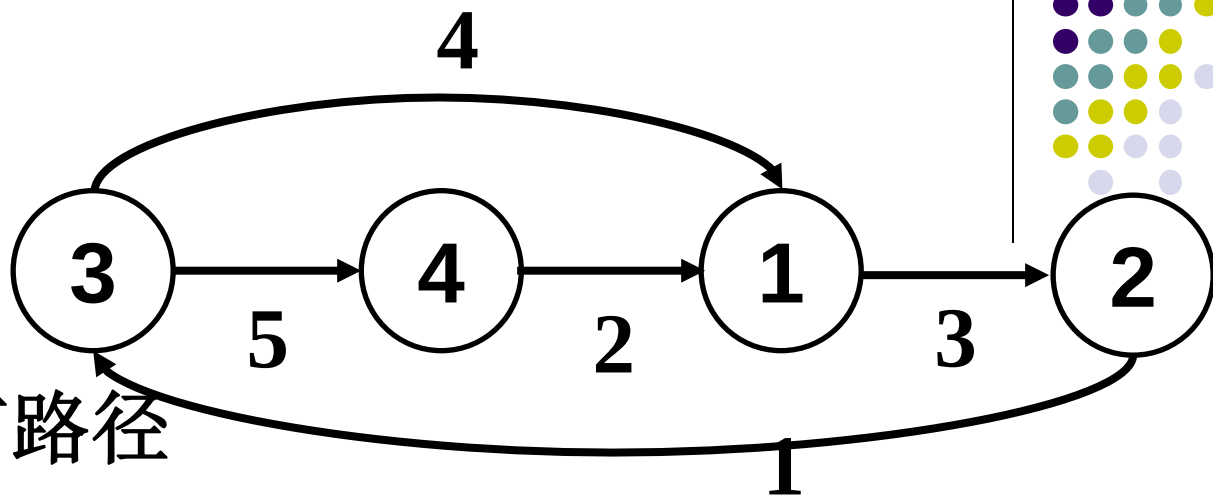
- 掌握Floyd算法
- 掌握图的传递闭包
- 了解Johnson算法
- 了解M最短路



# 任意两点间的最短路径问题

- 任意两点间最短路径问题，也称为多源最短路径问题。即对图中的每一对顶点  $v_i \neq v_j$ ，求  $v_i$  与  $v_j$  间的最短路径（最短路径长度）。
- 显然，最容易的想法就是：依次把每个顶点作为源点，**执行  $n$  次单源最短路径算法。**

# 枚举法



## □ 直接枚举所有路径

✓ 计算量大

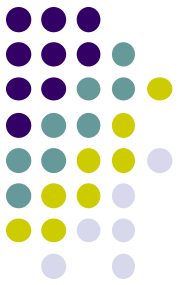
## □ 按长度枚举

✓ 处理复杂；路径有重复计算，效率也不高

## □ 路径拼接

✓ 唯一标识：标号最大点处拼接

✓ 设中间点标号集合： $S_0=\{ \}$ ,  $S_1=\{V_1\}$ ,  $S_2=\{V_1, V_2\}, \dots, S_n=\{V_1, V_2, \dots, V_n\}$



# Floyd算法

定义初始矩阵  $A^{(0)}[i][j] = \text{edge}[i][j]$ （图的邻接矩阵）；

1 求  $A^{(1)}$ , 即从  $v_i$  到  $v_j$  经由顶点可以是  $\{v_1\}$  的最短路径长度；

2 求  $A^{(2)}$ , 即从  $v_i$  到  $v_j$  经由顶点可以是  $\{v_1, v_2\}$  且在  $v_2$  处拼接的最短路径长度；

...

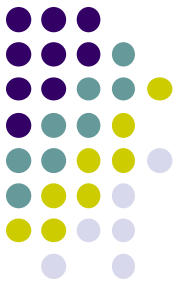
n 求  $A^{(n)}$ , 即从  $v_i$  到  $v_j$  经由顶点可以是  $\{v_1, v_2, \dots, v_n\}$  且在  $v_n$  处拼接的最短路径长度；

其中  $A^{(k)}[i][j] = \min \{ A^{(k-1)}[i][j],$

$$A^{(k-1)}[i][k] + A^{(k-1)}[k][j] \}, k = 1, 2, \dots, n$$



|   | $A^{(-1)}$    |          |          |    | $A^{(0)}$    |          |          |    | $A^{(1)}$    |          |    |    | $A^{(2)}$    |    |    |    | $A^{(3)}$    |    |    |    |
|---|---------------|----------|----------|----|--------------|----------|----------|----|--------------|----------|----|----|--------------|----|----|----|--------------|----|----|----|
|   | 0             | 1        | 2        | 3  | 0            | 1        | 2        | 3  | 0            | 1        | 2  | 3  | 0            | 1  | 2  | 3  | 0            | 1  | 2  | 3  |
| 0 | 0             | 1        | $\infty$ | 4  | 0            | 1        | $\infty$ | 4  | 0            | 1        | 10 | 3  | 0            | 1  | 10 | 3  | 0            | 1  | 9  | 3  |
| 1 | $\infty$      | 0        | 9        | 2  | $\infty$     | 0        | 9        | 2  | $\infty$     | 0        | 9  | 2  | 12           | 0  | 9  | 2  | 11           | 0  | 8  | 2  |
| 2 | 3             | 5        | 0        | 8  | 3            | 4        | 0        | 7  | 3            | 4        | 0  | 6  | 3            | 4  | 0  | 6  | 3            | 4  | 0  | 6  |
| 3 | $\infty$      | $\infty$ | 6        | 0  | $\infty$     | $\infty$ | 6        | 0  | $\infty$     | $\infty$ | 6  | 0  | 9            | 10 | 6  | 0  | 9            | 10 | 6  | 0  |
|   | $Path^{(-1)}$ |          |          |    | $Path^{(0)}$ |          |          |    | $Path^{(1)}$ |          |    |    | $Path^{(2)}$ |    |    |    | $Path^{(3)}$ |    |    |    |
|   | 0             | 1        | 2        | 3  | 0            | 1        | 2        | 3  | 0            | 1        | 2  | 3  | 0            | 1  | 2  | 3  | 0            | 1  | 2  | 3  |
| 0 | -1            | 0        | -1       | 0  | -1           | 0        | -1       | 0  | -1           | 0        | 1  | 1  | -1           | 0  | 1  | 1  | -1           | 0  | 3  | 1  |
| 1 | -1            | -1       | 1        | 1  | -1           | -1       | 1        | 1  | -1           | -1       | 1  | 1  | 2            | -1 | 1  | 1  | 2            | -1 | 3  | 1  |
| 2 | 2             | 2        | -1       | 2  | 2            | 0        | -1       | 0  | 2            | 0        | -1 | 1  | 2            | 0  | -1 | 1  | 2            | 0  | -1 | 1  |
| 3 | -1            | -1       | 3        | -1 | -1           | -1       | 3        | -1 | -1           | -1       | 3  | -1 | 2            | 2  | 3  | -1 | 2            | 0  | 3  | -1 |



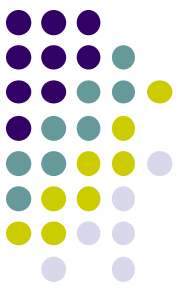
# Floyd算法的正确性

- 最优子结构（最优性原理）：最短路径中的任意一段子路径都是最短的；
- 递推：通过最短路径拼接

$$A^{(k)}[i][j] = \min \{ A^{(k-1)}[i][j],$$

$$A^{(k-1)}[i][k] + A^{(k-1)}[k][j] \}, k = 1, 2, \dots, n$$

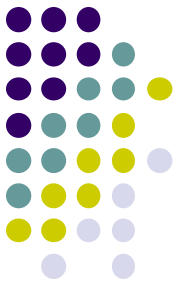
$$A^{(0)}[i][j] = \text{邻接矩阵}$$



# Folyd算法需要的空间

- 当以 $k$ 为拼接点时，矩阵的第 $k$ 行和第 $k$ 列的数值是不会改变的，相当于坐标；其它行和其它列的值如果改变，都是通过第 $k$ 行和第 $k$ 列的数值相加得到。
- 因此， $A$ 的计算从前到后都可以使用一个矩阵。





# Floyd算法描述

算法 Floyd ( )

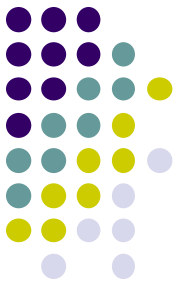
/\* 求每对顶点间的最短路径，其中 $\text{edge}[n][n]$ 表示有 $n$ 个顶点的图的邻接矩阵； $A[i][j]$ 表示顶点 $V_i$ 至 $V_j$ 的最短路径长度； \*/

F1[初始化]

for(  $i = 1 ; i \leq n ; i ++$  )

    for(  $j = 1 ; j \leq n ; j ++$  )

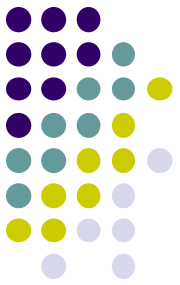
$A[i][j] = \text{edge}[i][j] ;$



**F2[求图中任意两顶点的最短路径]**

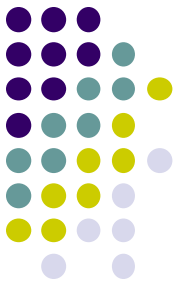
```
for(k=1; k<=n ; k++ )  
    for( i =1 ; i <= n ; i ++)  
        for( j=1 ; j<=n ;j++ )  
            if(A[i][k]+A[k][j]<A[i][j]))  
                A[i][j] = A[i][k]+A[k][j];
```





# path[i][j]

- **path[i][j]**表示相应路径上顶点j的前一个顶点的序号；
  - ✓ 初始化:  $\text{path}[i][j] = i \quad \langle i, j \rangle \in E$  ; otherwise -1
  - ✓ 递 推:  $\text{path}[i][j] = \text{path}[k][j]$  ;
- **path[i][j]**表示相应路径上顶点i的后一个顶点的序号；
- **path[i][j]**表示相应路径上顶点i和顶点j的最近一次中间点编号；



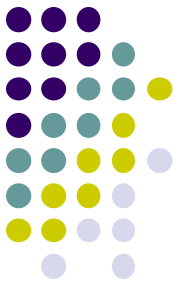
# Floyd算法小结

- Floyd算法的时间复杂度为 $O(n^3)$ 。
- Floyd算法的空间复杂度为 $O(n^2)$ 。
- Floyd算法是直接求任意两点间最短路的方法。
- 实现简单，主体是三重循环。
- Floyd算法可应用于负权边的情况。



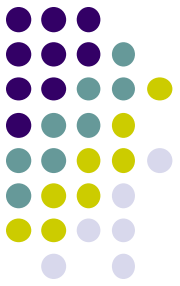
# Floyd算法 VS n次单源最短路

- N次BFS，时间效率 $n*(n+e)$ ，较好，适用无权
- n次Dijkstra，时间效率 $n*n^2$ ，效率相当，适用非负权
- n次Bellman-Ford，时间效率 $n*ne$ ，效率低
- n次SPFA，时间效率 $n*ne$ ，平均较好



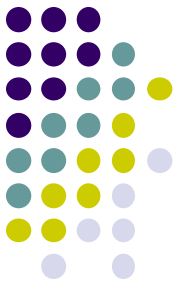
# Johnson算法的提出

- Floyd算法优雅
- Floyd效率较低，时间复杂度 $O(n^3)$
- N次dijkstra+堆优化较优，使用Fibonacci堆优化能做到 $n^2 \log n$ ，但不能处理负权。
- Johnson算法思想：将图转换成无负权的图，再使用dijkstra.



# 如何转换成无负权的图？

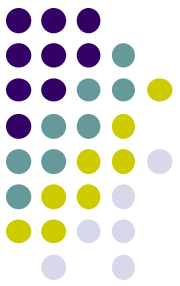
- ❑ 直接加最大负权的绝对值（减最大负权值）？
- ❑ 不正确！可以构造反例



# 重赋权(rewrite)

- 引入势能函数 $h[]$ ，对每一条边 $(u,v)$ ，其权值重写为  $W(u,v) = w(u,v) + h(u) - h(v)$
- 定理：重赋权不改变最短路径，而且，使用 $w$ 时不包含负权回路，当且仅当使用 $W$ 时也不包含负权回路。



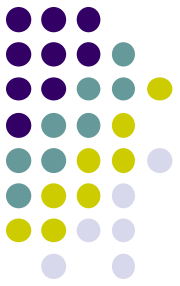


□ 证明：设  $p=(v_0, v_1, \dots, v_k)$  为从  $v_0$  到  $v_k$  的任意一条路径。则

$$\begin{aligned} W(p) &= \sum W(v_{i-1}, v_i) \\ &= \sum \{ w(v_{i-1}, v_i) + h(v_{i-1}) - h(v_i) \} \\ &= \sum w(v_{i-1}, v_i) + h(v_0) - h(v_k) \end{aligned}$$

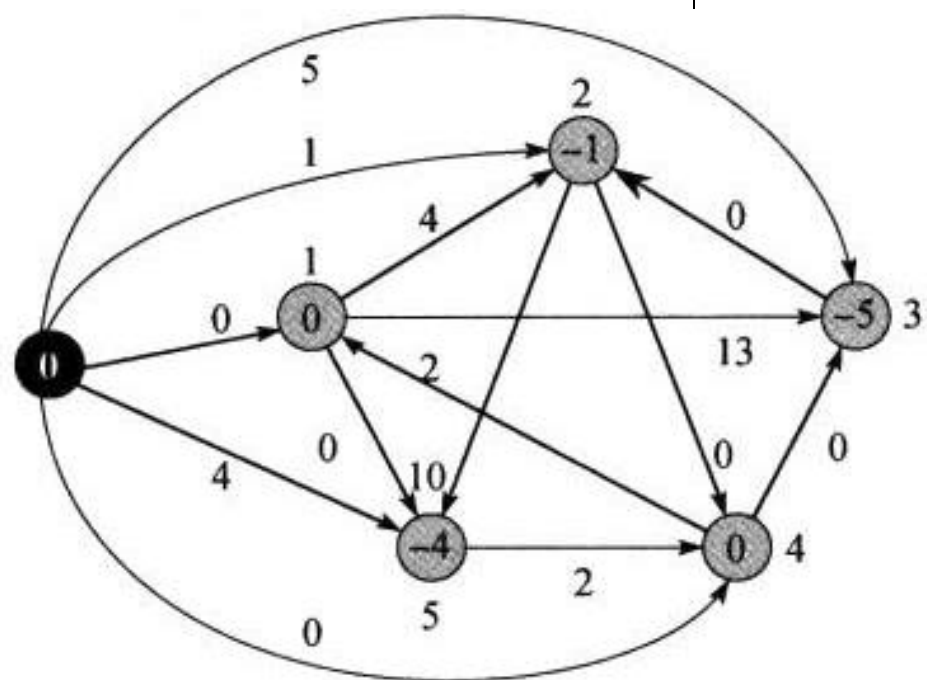
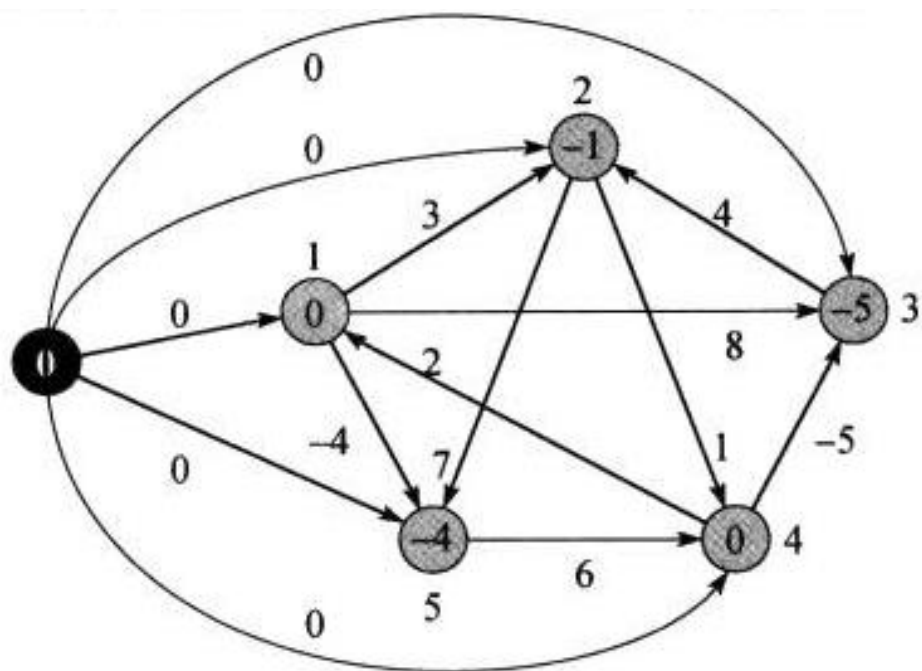
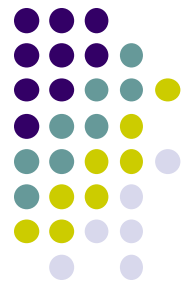
因为  $h(v_0)$  ,  $h(v_k)$  不依赖任何具体的路径，因此,  $w(p) = \delta(v_0, v_k)$  当且仅当  $W(p) = \delta(v_0, v_k)$

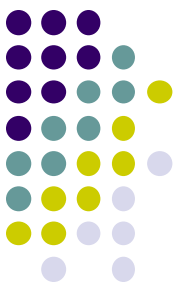
考虑任意环路  $c=(v_0, v_1, \dots, v_k)$ , 有  $v_0 = v_k$ , 从而  $W(c) = w(c)$ . 因此，环路  $c$  在  $w$  下为负当且仅当在  $W$  下为负



# 如何确定 $h$ ?

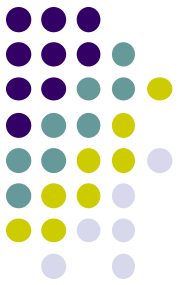
- 引入虚源点  $s$
- 令  $h(v) = \delta(s, v)$
- 从而，对于所有的边 $(u, v)$ ，  $W(u, v)$ 为非负





# Johnson算法描述

1. 对于给定图  $G=(V,E)$ ，新增一顶点  $S$ ，对  $S$  到图中所有点都建一条边，得到新图  $G'$
2. 对图  $G'$  中点  $S$  使用 Bellman-Ford 算法计算单源最短路，得到势能函数  $h[]$
3. 对原图  $G$  中所有的边进行重赋值：对于每条边  $(u,v)$ ，其新的权值为  $w(u,v)+h[u]-h(v)$
4. 对原图  $G$  的每个顶点运行 Dijkstra，求得全源最短路径



# Johnson算法描述

□ 算法Johnson( $G, w$ )

**J1.[新图 $G'$ ]**

$G'.V = G.V + \{s\}, G'.E = G.E + \{(s, v)\}, w(s, v) = 0;$

**J2.[重赋权]**

if ! Bellman-Ford( $G', w, s$ ) { print("n-cycle"); return; }

for each  $v \in G'.V$  {  $h[v] = \delta(s, v);$  }

for each  $(u, v) \in G'.E$ , {  $W(u, v) = w(u, v) + h[u] - h[v]$  }

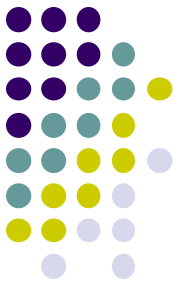
**J3.[n次单源最短路]**

for each  $u \in G.V$  {

    Dijkstra( $G, W, u$ );

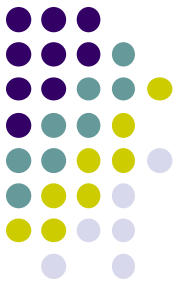
    for each  $v \in G.V$  {  $dis[u][v] = dis[u][v] + h[v] - h[u];$  }

}



# Johnson算法分析

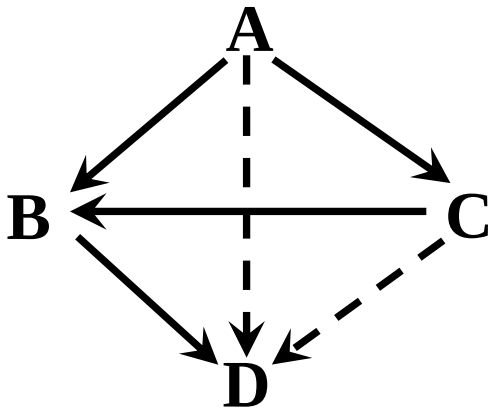
- 时间复杂度:  $O(n^3+n*e)$
- 优化措施: 对于无负环的边稠密图
  - ✓ 使用SPFA代替Bellman-Ford
  - ✓ 使用Fibonacci堆优化的Dijkstra
  - ✓ 优化后可达到 $O(n^2\log n)$



# 可及性

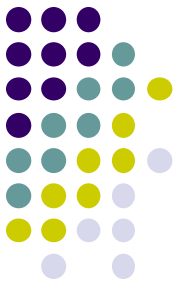
- 图 $G$ 中的两个顶点 $v_i$ 和 $v_j$ ，若从 $v_i$ 到 $v_j$ 存在一条有向路径，则称 $v_i$ 到 $v_j$ 可及。
- 用 $n \times n$ 阶可及矩阵 $R$ 来描述顶点之间的可及关系，可及矩阵 $R$ 的定义如下：

若 $v_i$ 到 $v_j$ 可及，则 $R_{ij}=1$ ，否则 $R_{ij}=0$ 。



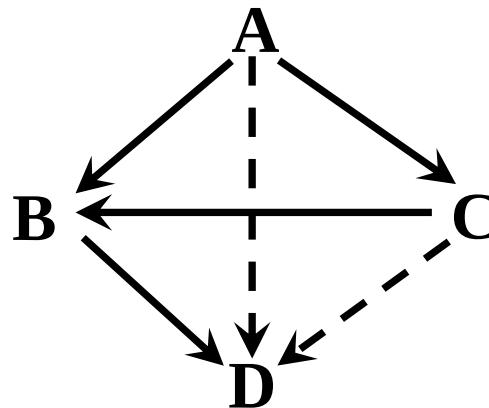
$$\begin{pmatrix} 1 & 1 & 1 & 1 \\ 0 & 1 & 0 & 1 \\ 0 & 1 & 1 & 1 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

可及矩阵

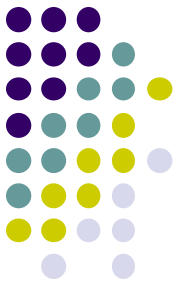


# 图的传递闭包

- 可及的传递性：若 $v_i$ 到 $v_j$ 可及， $v_j$ 到 $v_k$ 可及，则 $v_i$ 到 $v_k$ 可及。
- 由图 $G$ 的顶点集 $V$ 、边集 $E$ ，以及新添加的虚边（表示顶点可及）构成原图 $G$ 的扩展图，也称图 $G$ 的传递闭包。





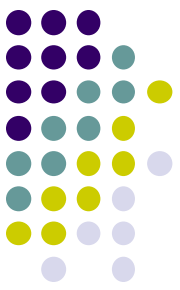


# Warshall算法

□  $WSM^{(k)}[i][j] =$   
 $WSM^{(k-1)}[i][j] \text{ OR}$   
 $WSM^{(k-1)}[i][k] \text{ AND } WSM^{(k-1)}[k][j], 1 \leq k \leq n$

其中,  $WSM^0=A$ ,  $WSM^{(k)}[i][j]$ 表示顶点  $i$  只经过顶点 $1, 2, \dots, k$  到达  $j$  的可及性.  $A$ 为有向图 $G$ 的邻接矩阵, 称 $WSM$ 为沃肖尔矩阵.

- 沃肖尔(Warshall) 算法与弗洛伊德(Floyd)算法类似, 是一个动态规划过程



# M最短路问题

- 按照路径长度的递增次序枚举出给定的两个结点间所有可能的简单路径
- 假设  $G=(V,E)$  是一个包含  $n$  个节点的有向图。集合  $P$  包含  $G$  中从  $v_1$  到  $v_n$  的所有简单路径，设  $p_1=r[0]$ ,  $r[1], \dots, r[k]$  是从  $v_1$  到  $v_n$  的一条最短路径，即， $p_1$  始于  $v_{r[0]} = v_1$ ，到  $v_{r[1]}$ ，然后到  $v_{r[2]}$ ,  $\dots$ ,  $v_{r[k]} = v_n$ 。



# 集合 $P - \{p_1\}$ 可划分成 $k$ 种情况

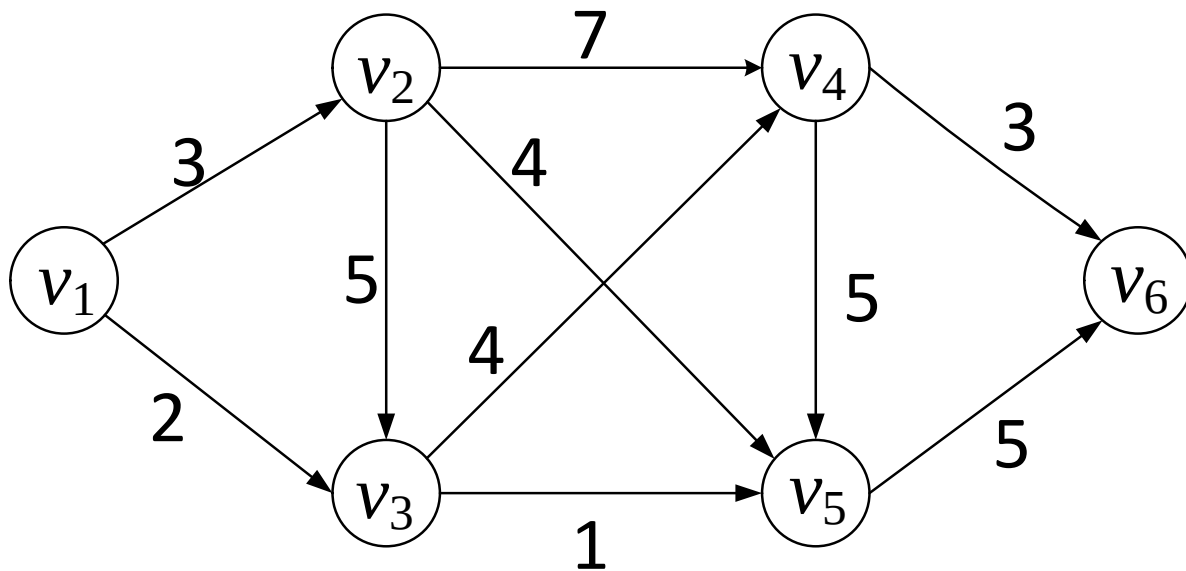
(k) 不包含边 $\langle v_{r[k-1]}, v_{r[k]} \rangle$ 。

(k-1) 包含边 $\langle v_{r[k-1]}, v_{r[k]} \rangle$ ，但不含 $\langle v_{r[k-2]}, v_{r[k-1]} \rangle$

.....

(2) 包含边 $\langle v_{r[2]}, v_{r[3]} \rangle, \dots, \langle v_{r[k-1]}, v_{r[k]} \rangle$ ，但是不含边 $\langle v_{r[1]}, v_{r[2]} \rangle$ ；

(1) 包含边 $\langle v_{r[1]}, v_{r[2]} \rangle, \dots, \langle v_{r[k-1]}, v_{r[k]} \rangle$ ，但是不包含边 $\langle v_{r[0]}, v_{r[1]} \rangle$ ；



| 最短路径     | 路径长度 | 包含的边                | 不包含的边   | 新的路径          |
|----------|------|---------------------|---------|---------------|
| v1v3v5v6 | 8    | 无                   | 无       |               |
|          |      | 无                   | <v5,v6> | v1v3v4v6=9    |
|          |      | <v5,v6>             | <v3,v5> | v1v2v5v6=12   |
|          |      | <v3,v5><br>,<v5,v6> | <v1,v3> | v1v2v3v5v6=14 |

# 求有向图前M条最短路算法

## MshortestPath( G, M)



MSP1[求从 $v_1$ 到 $v_n$ 的最短路径]

$Q \leftarrow \{(\text{从 } v_1 \text{ 到 } v_n \text{ 的最短路径}, \emptyset)\}.$

MSP2[求M条最短路径] //依次产生前M条最短路径

FOR  $i=1$  TO  $M$  DO (

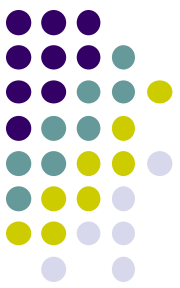
(1)令 $(p, C)$ 是 $Q$ 所有元素中 $p$ 长度最小的二元组.

(2)输出路径 $p$ , 并从 $Q$ 中删除 $p$ . // $p$ 是第 $i$ 短的最短路径

(3)在已有约束 $C$ 和划分产生的新约束下找出若干条 $G$ 中的最短路径.

(4)将这些最短路径及其约束构成的二元组添加到 $Q$ 中).





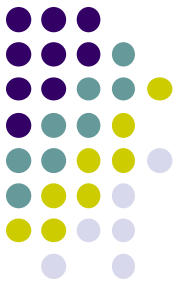
# M最短路算法分析

## □ 时间复杂度

- ✓ FOR循环中的第3行最多需要求出 $n - 1$ 条最短路径，在每轮迭代中求最短路径的时间复杂度为 $O(n^2)$ ，因此执行完FOR循环后这一行需要的总时间为 $O(Mn^3)$ 。
- ✓ For循环中的第一行和第四行，将 $Q$ 看作堆结构，则计算的时间复杂度会小于 $O(Mn^3)$ 。因此，生成 $M$ 条最短路径总的时间复杂度是 $O(Mn^3)$ 。

## □ 空间复杂度

- ✓ FOR循环的第三行每次迭代的时候至多会生成 $n-1$ 条最短路径，最多有 $O(Mn)$ 个元组存入 $Q$ 中。每条路径最多有 $O(n)$ 条边。因此，该算法的空间复杂度是 $O(Mn^2)$



# 最短路小结

## □ 单源最短路径

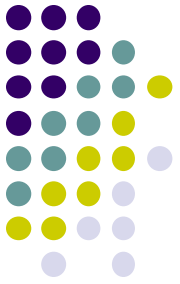
- ✓ 无权最短路径 (BFS)
- ✓ 正权最短路径 (Dijkstra)
- ✓ 负权最短路径: Bellman-Ford, SPFA

## □ 每对顶点之间的最短路径问题

- ✓ Floyd算法
- ✓ 图的传递闭包

## □ 拓展: M最短路、Johnson算法

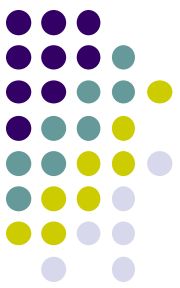
## □ 求解最短路径的算法, 有向图无向图均可。



# 思考

- 最长路问题
- 应用Floyd算法判负权环





# Interval

- 有一个序列，用 $n$ 个区间组合  $[a_i, b_i, c_i]$ 来描述它， $[a_i, b_i, c_i]$ 表示在该序列中处于 $[a_i, b_i]$ 这个区间的整数至少有 $c_i$ 个。如果存在这样的序列，请求出满足题目要求的最短的序列长度是多少。如果不存在则输出 **-1**。
- 输入：第一行包括一个整数 $n$ ，表示区间个数，以下 $n$ 行每行描述这些区间，第 $i+1$ 行三个整数 $a_i$ ， $b_i$ ， $c_i$ ，由空格隔开，其中 $0 \leq a_i \leq b_i \leq 50000$ 而且  $1 \leq c_i \leq b_i - a_i + 1$ 。
- 输出：一行，输出满足要求的序列的长度的最小值。

# 样例



## □ Sample Input

✓ 3

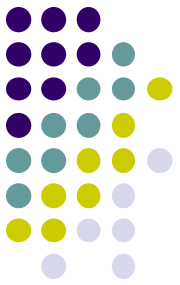
✓ 1 4 2

✓ 3 6 3

✓ 2 3 1

## □ Sample Output

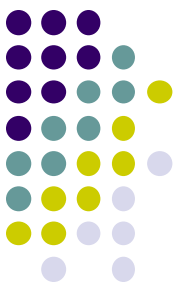
✓ 3



# 将问题数字化

- 定义数组 $\mathbf{t}$ ，若数字 $i$ 出现在序列中，则 $t_i = 1$ ，否则 $t_i = 0$ ，那么本题约束条件即为

$$\sum_{j=a_i}^{b_i} t_j \geq c_i \quad (i = 1, 2, \dots, n)$$



# 建立不等式模型

- 这样的描述使一个约束条件所牵涉的变量太多，不妨

设 
$$S_i = \sum_{j=1}^i t_j \quad (i=1,2,\dots,n)$$

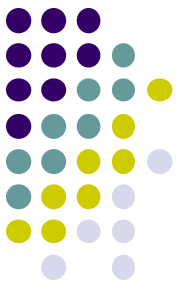
- 约束条件即可用以下不等式表示

$$S_{b_i} - S_{a_i-1} \geq c_i \quad (i = 1, 2, \dots, n)$$

- 注意：这样定义的 $\mathbf{S}$ 若仅仅满足约束条件的要求，不能完整体现它的意义； $\mathbf{S}$ 中的各个组成之间不是相对独立的，存在着联系。由于 $\mathbf{t}$ 数组要么为1要么为0，则 $\mathbf{S}_i$ 一定比 $\mathbf{S}_{i-1}$ 大，且至多大1，于是有

$$S_i - S_{i-1} \geq 0 \quad (i = 1, 2, \dots, n)$$

$$S_{i-1} - S_i \geq -1 \quad (i = 1, 2, \dots, n)$$



- 我们需要寻找一个满足以下要求的**S**数组

$$S_{b_i} - S_{a_i-1} \geq c_i \quad (i = 1, 2, \dots, n)$$

$$S_i - S_{i-1} \geq 0 \quad (i = 1, 2, \dots, n)$$

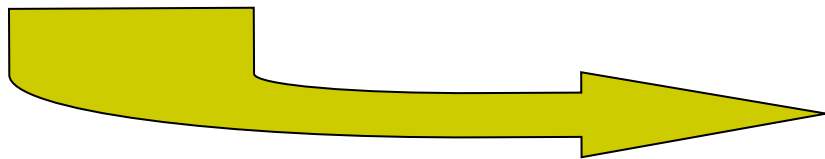
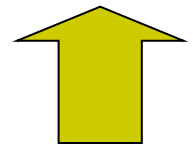
$$S_{i-1} - S_i \geq -1 \quad (i = 1, 2, \dots, n)$$

$$d[u] - d[v] \geq -w[u, v]$$



- 而**Bellman-Ford** 每次的更新操作为

If  $d[u] + w[u, v] \leq d[v]$  then  $d[v] \leftarrow d[u] + w[u, v]$



$$d[v] \leq d[u] + w[u, v]$$

# 例

## □ INPUT

3

1 4 2

3 6 3

2 3 1

$$S_4 - S_0 \geq 2$$

$$S_6 - S_2 \geq 3$$

$$S_3 - S_1 \geq 1$$

## □ 于是做出如下的转化:

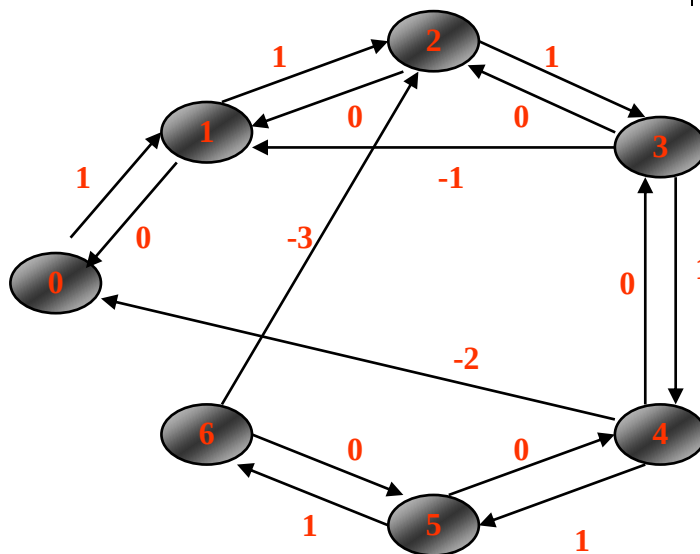
1. 我们将  $S_0, S_1, \dots, S_n$  看作  $n+1$  个点  $V_0, V_1, \dots, V_n$

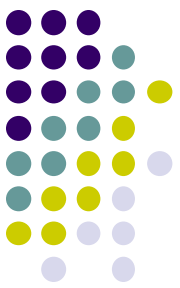
2. 对诸如  $A-B \geq C$  的形式, 我们从  $A$  向  $B$  连一条有向边, 权值为  $-C$ 。

1)  $S_{bi} - S_{ai-1} \geq C_i$

2)  $S_i - S_{i-1} \geq 0$

3)  $S_{i-1} - S_i \geq -1$





# 差分约束系统

$$x_1 - x_2 \leq 0$$

$$x_1 - x_5 \leq -1$$

$$x_2 - x_5 \leq 1$$

$$x_3 - x_1 \leq 5$$

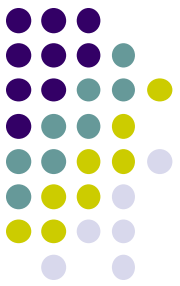
$$x_4 - x_1 \leq 4$$

$$x_4 - x_3 \leq -1$$

$$x_5 - x_3 \leq -3$$

$$x_5 - x_4 \leq -3$$

- 当 $x=(x_1, x_2, \dots, x_n)$ 是差分约束系统 $Ax \leq b$ 的一个解， $d$ 为任意常数。则 $x+d=(x_1+d, x_2+d, \dots, x_n+d)$ 也是该系统 $Ax \leq b$ 的一个解。因此这个不等式组要么无解，要么就有无数组解。



$$d[v] \leq d[u] + w(u,v)$$

$$\begin{array}{lcl} x_1 - x_2 & \leq & 0 \\ x_1 - x_5 & \leq & -1 \\ x_2 - x_5 & \leq & 1 \\ x_3 - x_1 & \leq & 5 \\ x_4 - x_1 & \leq & 4 \\ x_4 - x_3 & \leq & -1 \\ x_5 - x_3 & \leq & -3 \\ x_5 - x_4 & \leq & -3 \end{array}$$

